

U2 · Aula 7 — Automação E2E com Maestro

"Por que YAML para testes E2E mobile?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Fundamentos:** por que Maestro venceu
- **Parte 2 — Flows compostos:** comandos, fragmentos, env, JS
- **Parte 3 — Ecossistema:** Appium, cloud, flakiness, IA
- **Parte 4 — Prática guiada:** os exercícios que você vai fazer

Depois desta aula: screencast — eu fazendo os exercícios passo a passo.

Objetivos de aprendizagem

- **Explicar** por que E2E declarativo reduz flakiness
- **Escrever** flows Maestro com YAML (tap, assert, swipe)
- **Compor** suítes com runFlow, conditional e env
- **Escolher** entre Maestro e Appium por contexto
- **Automatizar** login e busca num app real (CineFav)

Parte 1 — Fundamentos

Por que Maestro · primeiro flow · comandos

Por que Maestro venceu (em adoção real)

Razões técnicas:

1. **YAML declarativo** vs código imperativo
2. **Retry built-in** elimina sleeps manuais
3. **Sem WebDriver Protocol** — fala direto com device via adb/simctl
4. **Setup em minutos**, não dias
5. **Cross-platform real** — mesmo flow iOS + Android
6. **Grátis** + cloud opcional (pago)

! Não é "killer" do Appium. É alternativa madura para o caso comum.

1. Declarativo vs imperativo

Imperativo = você escreve **COMO** (achar, esperar, clicar, asserir — e gerencia tudo).

Declarativo = você diz **O QUE** quer; o motor acha, espera e re-tenta sozinho.

✗ Imperativo (Appium / WebdriverIO)

```
await driver.$('~email')
  .waitForDisplayed({ timeout: 5000 });
await driver.$('~email').setValue('a@b.com');
await driver.$('~submit').click();
await expect(driver.$('~home'))
  .toBeDisplayed();
```

✓ Declarativo (Maestro)

```
- tap0n: { id: "email" }
- inputText: "a@b.com"
- tap0n: { id: "submit" }
- assertVisible: { id: "home" }
```

Mesmo resultado, metade das linhas. QA foca no **fluxo**, não na mecânica de espera/clique.

2. Retry built-in (adeus `sleep`)

A tela demora pra carregar (rede, animação). Como esperar o elemento?

✗ **Sem retry** — você chuta o tempo

```
await sleep(2000); // "2s deve dar"
await driver.$('~ok').click();
```

Curto demais → falha (flaky).

Longo demais → suíte lenta.

✓ **Maestro** — re-tenta sozinho

```
tapOn: { id: "ok" }
```

O motor procura/toca **repetidamente** até o elemento aparecer (ou estourar o timeout). **Sem `sleep`**.

`sleep` fixo é a **causa #1** de teste flaky. Retry automático = estável sem você pensar nisso.

3 + 5. Sem WebDriver = cross-platform de verdade

WebDriver é um protocolo HTTP/JSON. O Appium põe um **servidor no meio**:

seu teste → servidor Appium → driver (UiAutomator2 / XCUITest) → device.

Cada plataforma tem **driver + capabilities** próprios → setup e seletores divergem.

Maestro fala **direto** com o device: adb (Android) · idb /simulator (iOS). O **mesmo YAML** roda nos dois — o motor traduz pra cada plataforma.

	Appium	Maestro
Camada	WebDriver server + driver	adb / idb direto
Flow iOS vs Android	costuma divergir	mesmo arquivo
Setup	drivers + capabilities	binário único

Por isso **"cross-platform real"**: 1 flow, 2 plataformas — sem reescrever. (Appium é cross-platform "no papel", mas na prática você mantém dois conjuntos de seletores.)

Quem usa Maestro em produção

Casos documentados:

Empresa	Contexto
Cash App (Block)	Reescreveu suíte iOS+Android completa
Reddit	Flows cross-platform únicos iOS+Android
Nubank	Mobile E2E de pagamentos (Brasil)
Grab	Super-app — 400+ flows em produção
Lingvano	React Native + Expo + Maestro — caso no Medium

Referências verificadas:

- Lingvano Eng. (2023) *"Native E2E testing with Expo, RN and Maestro"* — Medium
- LambdaTest (2024) *"Future of QA"* — survey 1.600+ QAs
- Linares-Vásquez et al. (2017) ICSME — base acadêmica automação mobile
- Wang et al. (2023) *"Empirical Study on Flaky Tests in Mobile Apps"* ASE 2023

IA + Maestro (2024–25):

- `maestro ai "descrição em PT"` → gera YAML automaticamente (beta)
- LLMs atualizam flows quando UI muda

Maestro — primeiro flow

```
# login_flow.yaml
appId: com.myapp
---
- launchApp
- tap0n: "Email"
- inputText: "user@email.com"
- tap0n: "Senha"
- inputText: "secret"
- tap0n: "Entrar"
- assertVisible: "Bem-vindo"
```

```
maestro test login_flow.yaml
```

Em 30 segundos você tem teste E2E rodando.

Maestro — comandos principais

```
- launchApp                # abre app
- tapOn: "Submit"          # tap por texto
- tapOn:
  id: "submit-button"      # tap por ID (preferido)
- inputText: "hello"        # digita
- swipe:
  start: "50%, 80%"
  end: "50%, 20%"
- scroll
- assertVisible: "Welcome"
- assertNotVisible: "Loading"
- pressKey: "BACK"
- waitForAnimationToEnd
- takeScreenshot: "after-login"
```

Parte 2 — Flows compostos

Conditional · fragmentos · env · JS · Studio

Maestro — conditional flows

```
appId: com.myapp
---
- launchApp
- runFlow:
  when:
    visible: "Aceitar cookies"
    file: ./flows/dismiss-cookies.yaml
- runFlow:
  when:
    notVisible: "Welcome"
    file: ./flows/login.yaml
- assertVisible: "Welcome"
```

Suíte se adapta ao estado do app. Sem `if` imperativo.

Flow vs Fragmento

Tecnicamente **são o mesmo tipo de arquivo YAML**. A diferença é estrutural:

✓ FLOW (completo)

tem `appId` + `---`

```
# tests/checkout.yaml
appId: com.myapp
---
- launchApp
- runFlow: ../flows/login.yaml
- tap0n: "Carrinho"
- assertVisible: "Pedido confirmado"
```

Roda standalone:

```
maestro test checkout.yaml
```

📦 FRAGMENTO (reutilizável)

também tem `appId` + `---`

```
# _fragments/login.yaml
appId: com.myapp
---
- tap0n: { id: "login-email-input" }
- inputText: "user@email.com"
- tap0n: { id: "login-submit-button" }
```

Você o chama de outro flow:

```
runFlow: _fragments/login.yaml
```

⚠ No Maestro 2.6 **todo** arquivo precisa de `appId` + `---` (config section) — inclusive fragmentos. "Fragmento" é só um flow que você reaproveita via `runFlow`. Guarde em `_fragments/` pra separar dos flows principais.

Maestro — parametrização via env

```
# tests/login.yaml
appId: com.myapp
env:
  EMAIL: "default@email.com"
  PASSWORD: "secret"
---
- launchApp
- inputText: ${EMAIL}
- inputText: ${PASSWORD}
```

```
maestro test login.yaml -e EMAIL=admin@myapp.com -e PASSWORD=adminPass
```

Reutiliza flow para múltiplas personas.

Maestro — JavaScript embutido no flow

output é um objeto GLOBAL que o Maestro injeta — você não declara, não importa. O que você grava em `output.x`, o YAML lê com `${output.x}`. (Runtime próprio, **não é Node** — sem `require`, sem `export`.)

A) inline — `evalScript: ${...}`

```
appId: com.myapp
---
- launchApp
- evalScript: ${output.email = `user-${Date.now()}@x.com`}
- inputText: ${output.email}
- tap0n: "Cadastrar"
```

B) arquivo — `runScript: arquivo.js`

```
- runScript: gera-dados.js
- inputText: ${output.email}
```

```
// gera-dados.js — SEM export, só atribui a output:
output.email = `user-${Date.now()}@x.com`;
```

Maestro roda o arquivo no mesmo runtime → o `output` é compartilhado com o flow.

`output.email` não vem de lugar nenhum que você importa — é o Maestro que cria o objeto `output` e injeta nos dois (script e flow).

tap0n: text vs tap0n: id — use id sempre que possível

✗ Frágil:

```
– tap0n: "Cadastrar"
```

Quebra em: tradução, A/B test, refactor de texto.

✓ Estável:

```
– tap0n:  
  id: "login-submit-button"
```

Independente de texto. App bem instrumentado expõe testID em todos os elementos críticos.

Regra: tap0n: id: sempre que o app expõe testID. tap0n: "texto" só se não tiver ID.

Maestro Studio — o que é

Ferramenta visual que vem com o CLI. Você **clica no app** e ela escreve o YAML.

```
maestro studio  # abre http://localhost:9999 no browser
```

O que você vê no Studio:

- Espelho ao vivo do simulador/device
- Clicou → gerou `tap0n:` automaticamente
- Digitou → gerou `inputText:`
- Inspetor de elementos (IDs, textos)
- Botão "Add assertion" → gera `assertVisible:`

Fluxo:

1. Simulador rodando com o app aberto
2. `maestro studio` no terminal
3. Browser abre → você interage no espelho
4. YAML vai sendo gerado ao lado
5. Salva → `maestro test flow.yaml`

Não precisa saber YAML pra começar. Studio é o ponto de entrada.

Parte 3 — Ecossistema e mercado

Appium · cloud devices · flakiness · IA

Maestro vs Appium — comparativo

Critério	Maestro	Appium 2
Linhas (mesmo flow)	~10	~30+
Tempo execução	~3s	~7–12s
Flake rate inicial	< 5%	~15–25%
Setup	10 min	1–2h
Cross-platform (mesmo YAML)	✓	✗
Apps nativos	✓	✓
Apps RN	✓	⚠ flake

Appium 2 — quando ainda usar

Não morreu. Casos onde Appium 2 ainda faz sentido:

Situação	Por quê Appium
Apps híbridos legados (WebView complexo)	Maestro não inspeciona WebView profundo
Selenium Grid existente na empresa	Custo de migração alto
Drivers especializados (Espresso, XCUITest, Mac)	Appium tem plugins específicos
Empresas com investimento Appium grande	Migração tem custo real

```
// Flow equivalente Appium 2 (TypeScript)
await driver.$('~email-field').setValue('user@email.com');
await driver.$('~submit-button').click();
expect(await driver.$('~welcome-text').isDisplayed()).toBe(true);
```

Maestro é mais rápido pra começar. Appium é mais poderoso em edge cases. Cada projeto avalia.

Cloud Devices — opções de mercado

Plataforma	Free tier	Forte em
Firebase Test Lab	100min/mês virtual + 5h/mês físico	Apps Android, integração Google
Maestro Cloud	✗ Pago	Flows YAML direto na nuvem
BrowserStack App Automate	Pago	Cross-platform em escala
LambdaTest	30min trial	Alternativa mais barata que BrowserStack
AWS Device Farm	1000min device-min free	Ecossistema AWS

| Cloud testing é **contexto de mercado** — o essencial é dominar flows rodando local + CI.

Flakiness mobile — causas e mitigações

Causa	Mitigação Maestro
Animações	<code>waitForAnimationToEnd</code>
OS popups (permissões)	<code>runFlow</code> <code>when: visible:</code> para dispensar
Texto que muda (A/B test, tradução)	<code>tap0n: id:</code> em vez de texto
Biometria	Mock no build de test
Network jitter	Mock server local
Lifecycle (background→foreground)	Estabilizar antes de <code>assert</code>
Touch latency em emulador	Confirmar em device real se flaky

Wang et al. (2023) ASE — estudo empírico sobre flaky tests em apps mobile. **iDFlakies** (Lam et al. 2019) catalogou causas — referências acadêmicas da área.

Pitfalls comuns

- ✗ `tapOn:` por texto que muda — flake em A/B test ou tradução; use `id:`
- ✗ Não usar `runFlow` para login — repetição em todos os testes
- ✗ Suíte sem retry — flake transitório vira falha permanente
- ✗ Executar só em emulator — esconde issues de touch latency
- ✗ Não monitorar flake rate — métrica essencial degrada silenciosamente

IA aplicada — gerar flow com Claude API

```
const userStory = "Como user, quero comprar produto e ver toast de confirmação.";

const response = await anthropic.messages.create({
  model: 'claude-opus-4-8',
  max_tokens: 1024,
  messages: [{
    role: 'user',
    content: `Gere flow Maestro YAML para:\n${userStory}\n` +
      `Use runFlow para fragmentos. Inclua conditional se necessário.`,
  }],
});
```

IA é boa em **transformação** (história → flow). Sempre revisar antes de aceitar.

Resumo

- **Maestro** = melhor padrão E2E mobile em 2026 (simplicidade + velocidade)
- **YAML declarativo + retry built-in** → menos código, menos flake
- **runFlow** → DRY; **conditional** → robustez frente a estado variável
- **tap0n: id:** sempre que houver testID — texto quebra em tradução/A/B
- **Cloud devices** = contexto de mercado; o essencial é local + CI

Parte 4 — Prática guiada

Os exercícios que você vai fazer (com screencast)

A prática — app CineFav

- **App de filmes real** — login, busca, favoritos, detalhe
- **Instrumentado** — `testID` em todos os elementos críticos
- **Repo com flows-guia** — TODOs marcando o que você completa
- **Dados mockados** — roda offline, sem token, determinístico
- **Screencast** — eu faço junto com você, passo a passo

Setup em 3 passos

```
# 1. Maestro CLI
curl -Ls get.maestro.mobile.dev | bash

# 2. app no device/emulador (APK no repo)
adb install CineFav.apk

# 3. rodar o primeiro flow
maestro test flows/01-launch.yaml
```

Guia completo por sistema operacional no repo da prática.

Prática 1 — Login (01-launch.yaml)

```
appId: com.puciec.cinefav
---
- launchApp
- tap0n: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tap0n: { id: "login-password-input" }
- inputText: "1234"
- tap0n: { id: "login-submit-button" }
- assertVisible: { id: "movielist-screen" }
- assertVisible: "Matrix"
```

Modelo resolvido — no screencast eu rodo e explico linha a linha.

Prática 2 — Busca (02-search.yaml)

```
env:  
  SEARCH_TERM: "Matrix"  
---  
- launchApp  
# TODO: faça login (copie do flow 01)  
# TODO: abra a busca - id "movielist-search-button"  
# TODO: inputText com ${SEARCH_TERM}  
# TODO: assertVisible do ${SEARCH_TERM}
```

Você completa os TODOs — busca parametrizada via env.

Prática 3 — Favoritar (`03-favorite.yaml`)

- **Fluxo cross-tela** — o teste atravessa 2 telas
- Favoritar no card — `movie-card-heart-{id}`
- Abrir Favoritos — `movielist-favorites-button`
- Conferir que o filme aparece em `favorites-screen`

Asserção de estado que **persiste na navegação**.

Prática 4 — Detalhe + JS (04-detail e 05-js-dynamic)

```
# 05: JS sorteia o filme buscado
- evalScript: ${output.termo = ['Matrix','Avatar','Interestelar'][Math.floor(Math.random() * 3)]}
- inputText: ${output.termo}
- assertVisible: ${output.termo}
```

- **Bônus:** extrair `_fragments/login.yaml` + `runFlow` em todo flow (o DRY da Parte 2)

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Repo da prática** — link no Canvas, flows-guia prontos
- **Seu ritmo** — pause, rode, compare com o resultado
- **Travou?** — Maestro Studio inspeciona os IDs ao vivo

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith